Angular 14 Practice Notes

# Q7-Q60 Concepts Explanation

This PDF explains every concept used in your separated Angular components. Each question includes the concept, why it is used, and the important syntax to remember for viva, practicals, and revision.

**Project:** angular-14-crud-example

# Component Communication and Binding

Q7

## Parent to child input binding

**Concept used:** @Input()

**Explanation:** The child component declares an input property using @Input(). The parent sends data using property binding. This is used when a child must display or use a value owned by the parent.

**Important syntax:**

```
[message]="parentMessage"
```

Q8

## Child to parent event binding

**Concept used:** @Output() and EventEmitter

**Explanation:** The child emits an event using EventEmitter. The parent listens to that event and receives the value through $event. This is used for button clicks, selected values, and child actions.

**Important syntax:**

```
(messageSent)="onChildMessage($event)"
```

Q9

## Disable button when input is empty

**Concept used:** Property binding

**Explanation:** The disabled property of a button should be controlled by a boolean condition. trim() is useful because it treats spaces as empty input.

**Important syntax:**

```
[disabled]="!name.trim()"
```

Q10

## Avoid repeated function calls in template

**Concept used:** Precomputed property

**Explanation:** Angular runs template expressions many times during change detection. Calling methods in templates can reduce performance. Calculate the value once in TypeScript and bind to a property.

**Important syntax:**

```
{{ total }}
```

Q11

## Apply class dynamically

**Concept used:** ngClass

**Explanation:** ngClass applies CSS classes based on conditions. It is useful when styles depend on values such as marks, status, or selected state.

**Important syntax:**

```
[ngClass]="{ pass: marks >= 50, fail: marks < 50 }"
```

Q12

## Fix multiple click events

**Concept used:** Event binding

**Explanation:** Each button should call only the method related to its action. Separate handlers make the template clear and avoid accidental conflicts.

**Important syntax:**

```
(click)="add()"
```

# Structural Directives

## Q13 Render users using a loop

**Concept used:** *ngFor

**Explanation:** *ngFor repeats an element for every item in an array. If nothing displays, check that the array exists and the directive syntax is correct.

**Important syntax:**

```
*ngFor="let user of users"
```

## Q14 Improve list rendering

**Concept used:** trackBy

**Explanation:** trackBy tells Angular how to uniquely identify list items. It improves performance because Angular can reuse DOM elements instead of recreating them.

**Important syntax:**

```
*ngFor="let user of users; trackBy: trackById"
```

## Q15 Display only one section

**Concept used:** *ngIf else

**Explanation:** Use *ngIf with an else template to make sure only one block is rendered at a time.

**Important syntax:**

```
*ngIf="isLoggedIn; else guest"
```

## Q16 Show loading until data arrives

**Concept used:** Conditional rendering

**Explanation:** A loading variable controls the UI. Show the loading message while data is not ready, then show the real content.

**Important syntax:**

```
*ngIf="loading; else dataBlock"
```

## Q17 Fallback for empty table

**Concept used:** *ngIf and ng-template

**Explanation:** Before rendering rows, check whether the data array has items. If the array is empty, show a friendly fallback message.

**Important syntax:**

```
*ngIf="rows.length; else emptyTable"
```

Q18

## Show fields based on dropdown

**Concept used:** *ngIf with ngModel

**Explanation:** The selected dropdown value is stored in a property. Extra fields appear only when the selected value matches the condition.

**Important syntax:**

```
*ngIf="type === 'student'"
```

Q19

## Render nested lists

**Concept used:** Nested *ngFor

**Explanation:** Use one loop for parent data and another loop inside it for child arrays. Each loop should have its own variable.

**Important syntax:**

```
*ngFor="let employee of department.employees"
```

Q20

## Avoid multiple structural directives on one element

**Concept used:** ng-container

**Explanation:** Angular allows only one structural directive on a single element. Use ng-container as a wrapper when *ngFor and *ngIf are both needed.

**Important syntax:**

```
<ng-container *ngFor="let user of users">
```

Q21

## Display first five items

**Concept used:** slice pipe

**Explanation:** The slice pipe shows part of an array without changing the original array in TypeScript.

**Important syntax:**

```
items | slice:0:5
```

Q22

## Highlight selected items

**Concept used:** Conditional styling

**Explanation:** Store whether an item is selected, then apply a CSS class only when selected is true.

**Important syntax:**

```
[ngClass]="{ selected: item.selected }"
```

# Template Driven Forms

Q23

## Capture user input

**Concept used:** FormsModule and ngModel

**Explanation:** Template-driven forms use [(ngModel)] for two-way binding. Every input must have a name attribute so Angular can track it.

**Important syntax:**

```
name="email" [(ngModel)]="user.email"
```

Q24

## Prevent invalid form submit

**Concept used:** required and form.invalid

**Explanation:** Required fields make the form invalid until filled. The submit button can be disabled using the form invalid state.

**Important syntax:**

```
[disabled]="userForm.invalid"
```

Q25

## Show errors only after touch

**Concept used:** touched state

**Explanation:** Validation messages should not appear immediately on page load. Use touched so errors show only after the user interacts with the field.

**Important syntax:**

```
*ngIf="nameCtrl.invalid && nameCtrl.touched"
```

Q26

## Validate email format

**Concept used:** email validator

**Explanation:** The email validator checks whether the typed value has a valid email format. Combine it with required when the field must not be empty.

**Important syntax:**

```
required email #emailCtrl="ngModel"
```

# Reactive Forms

## Q27 Initialize form controls

**Concept used:** FormBuilder

**Explanation:** Reactive forms are created in TypeScript. FormBuilder makes it easier to create FormGroup objects and validators.

**Important syntax:**

```
this.fb.group({ name: ['', Validators.required] })
```

## Q28 Validate password length

**Concept used:** Validators.minLength

**Explanation:** Use minLength for passwords or fields that need a minimum number of characters. The template can check hasError('minlength').

**Important syntax:**

```
Validators.minLength(6)
```

## Q29 Reset after submit

**Concept used:** form.reset()

**Explanation:** After successful submission, call reset() to clear all fields and restore the form state.

**Important syntax:**

```
this.form.reset()
```

## Q30 Capture all field values

**Concept used:** FormGroup value

**Explanation:** Every field must be part of the FormGroup and connected with formControlName. Then form.value contains all entered values.

**Important syntax:**

```
this.form.value
```

## Q31 Conditionally required field

**Concept used:** Dynamic validators

**Explanation:** A field can become required based on another field's value. Add or clear validators and call updateValueAndValidity().

**Important syntax:**

```
control.setValidators([Validators.required])
```

## Q32 Dropdown updates another field

**Concept used:** valueChanges

**Explanation:** valueChanges listens to control changes. When the dropdown changes, patchValue can update another control automatically.

**Important syntax:**

```
this.form.patchValue({ city: 'Pune' })
```

## Q33 Add repeated fields

**Concept used:** FormArray

**Explanation:** FormArray is used for dynamic controls such as skills, phone numbers, or addresses. push() adds a new control.

**Important syntax:**

```
this.skills.push(this.fb.control(''))
```

## Q34 Use one form approach

**Concept used:** Consistent form style

**Explanation:** Do not mix ngModel and reactive forms in the same form. Choose template-driven or reactive forms for one form.

**Important syntax:**

```
[formGroup]="form"
```

## Q35 Hide validation on load

**Concept used:** touched with reactive forms

**Explanation:** Reactive validation messages should also check touched or dirty so they appear only after interaction.

**Important syntax:**

```
control.invalid && control.touched
```

## Q36 Disable submit until valid

**Concept used:** form.invalid

**Explanation:** The full form validity can control the submit button. This prevents sending incomplete or invalid data.

**Important syntax:**

```
[disabled]="form.invalid"
```

# Services, HTTP and State

## Q37 Move API logic to service

**Concept used:** Angular service

**Explanation:** Components should handle UI. Services should handle reusable data and API logic. This keeps code clean and testable.

**Important syntax:**

```
constructor(private dataService: PracticeDataService) {}
```

## Q38 Fetch data from API

**Concept used:** HttpClient.get

**Explanation:** HttpClient.get() sends a GET request and returns an Observable. HttpClientModule must be imported in AppModule.

**Important syntax:**

```
this.http.get<User[]>(url)
```

## Q39 Render API response

**Concept used:** subscribe and *ngFor

**Explanation:** After subscribing to API data, store the response in an array property. The template displays it using *ngFor.

**Important syntax:**

```
users = response
```

## Q40 Send POST payload

**Concept used:** HttpClient.post

**Explanation:** POST requests need a URL and payload object. The payload contains the data sent to the server.

**Important syntax:**

```
this.http.post(url, userData)
```

## Q41 Update UI after delete

**Concept used:** Refresh local data

**Explanation:** After deleting, remove the item from the displayed array or re-fetch the data. The UI changes only when component data changes.

**Important syntax:**

```
users.filter(user => user.id !== id)
```

Q42

## Share data across components

**Concept used:** BehaviorSubject

**Explanation:** BehaviorSubject stores the latest value and emits it to all subscribers. It is useful for shared state.

**Important syntax:**

```
users$ = subject.asObservable()
```

Q43

## Fetch on initialization

**Concept used:** ngOnInit

**Explanation:** ngOnInit is the correct place to load data when a component starts. Constructor should mainly be used for dependency injection.

**Important syntax:**

```
ngOnInit(): void { this.loadData(); }
```

Q44

## Handle API errors

**Concept used:** catchError

**Explanation:** catchError prevents the app from crashing. It lets you display a message and return fallback data.

**Important syntax:**

```
catchError(() => of([]))
```

Q45

## Avoid stale data after update

**Concept used:** Update or re-fetch

**Explanation:** After updating data, update the local list or load the data again so the screen shows the latest value.

**Important syntax:**

```
this.loadUsers()
```

Q46

## Remove duplicate API code

**Concept used:** Central service method

**Explanation:** If many components use the same API logic, move it to one service method and call that method from components.

**Important syntax:**

```
dataService.getUsers()
```

Q47

## Show loading during API call

**Concept used:** loading flag and finalize

**Explanation:** Set loading true before the API call. Use finalize() to set loading false after success or failure.

**Important syntax:**

```
finalize(() => this.loading = false)
```

Q48

## Avoid repeated API calls

**Concept used:** Do not call APIs in template

**Explanation:** Methods used in templates can run many times. API calls should be done in ngOnInit or event handlers, not directly in interpolation.

**Important syntax:**

```
Load once in ngOnInit
```

# Routing

## Q49 Configure routing

**Concept used:** RouterModule

**Explanation:** Routes map URL paths to components. RouterModule.forRoot(routes) enables routing in the application.

**Important syntax:**

```
{ path: 'question/:id', component: Q51RouteParamComponent }
```

## Q50 Avoid page reload

**Concept used:** routerLink

**Explanation:** Use routerLink instead of href for Angular navigation. routerLink changes route without refreshing the whole page.

**Important syntax:**

```
routerLink="/question/50"
```

## Q51 Read route parameter

**Concept used:** ActivatedRoute

**Explanation:** ActivatedRoute gives access to route parameters. paramMap reads values from URLs like /question/51.

**Important syntax:**

```
params.get('id')
```

## Q52 Navigate from TypeScript

**Concept used:** Router.navigate

**Explanation:** Programmatic navigation is used after actions such as successful form submission.

**Important syntax:**

```
this.router.navigate(['/submitted'])
```

## Q53 Handle unknown URLs

**Concept used:** Wildcard route

**Explanation:** The wildcard path catches all unknown routes and redirects to a safe page.

**Important syntax:**

```
{ path: '**', redirectTo: '' }
```

Q54

## Render child route

**Concept used:** Nested router-outlet

**Explanation:** Child routes need a router-outlet inside the parent component template. The child component appears inside that outlet.

**Important syntax:**

```
<router-outlet></router-outlet>
```

Q55

## Pass query parameters

**Concept used:** queryParams

**Explanation:** Query parameters are optional URL values like ?filter=active. They are useful for filters, search text, and page numbers.

**Important syntax:**

```
{ queryParams: { filter: 'active' } }
```

# Pipes and Mini App

Q56

## Format date

**Concept used:** date pipe

**Explanation:** The date pipe changes a raw date into a readable format without modifying the original value.

**Important syntax:**

```
today | date:'mediumDate'
```

Q57

## Display uppercase names

**Concept used:** uppercase pipe

**Explanation:** The uppercase pipe changes text display only. The original data remains unchanged.

**Important syntax:**

```
name | uppercase
```

Q58

## Truncate long text

**Concept used:** slice pipe

**Explanation:** The slice pipe can shorten long strings in the UI. Add three dots after it for an ellipsis effect.

**Important syntax:**

```
text | slice:0:45
```

Q59

## Format currency

**Concept used:** currency pipe

**Explanation:** The currency pipe displays numbers as money with the selected currency symbol.

**Important syntax:**

```
price | currency:'INR':'symbol'
```

Q60

## Fix mini app

**Concept used:** Combined Angular concepts

**Explanation:** The mini app combines list rendering, form validation, service data, and routing. Each feature must be correctly connected for the app to work.

**Important syntax:**

```
form + *ngFor + service + routerLink
```

Generated for Angular Q7-Q60 component practice.